UNIVERSITY TIMETABLE OPTIMIZATION SYSTEM

Software Engineering Project Document

Process Model · Requirements · Use Cases · System Modeling

Project: Optimize Flow

Version 1.0

Table of Contents

1. Introduction

1.1 What is this System?

This system is a web-based university timetable optimization platform that helps create, manage, and improve class schedules for teachers, students, rooms, and courses. It automatically generates timetables by following important rules such as teacher availability, room capacity, class conflicts, holidays, and student sections, while also considering preferences like reducing travel between classes, saving electricity, ending classes earlier, and making timetable changes with minimum disturbance.

1.2 Problem Solved

It solves the problem of manually creating university timetables, which is time-consuming, error-prone, and difficult when there are many teachers, classes, rooms, holidays, and restrictions. The system helps avoid clashes, uses rooms properly, respects teacher availability, and makes timetable changes easier without disturbing the whole schedule.

1.3 Scope

Explicitly not in scope are exam timetabling, bus or transport route management, hostel scheduling, individual personalized timetables for every student, attendance management, fee management, full university ERP features, and advanced AI prediction. The system will focus only on generating, managing, and adjusting weekly class timetables for teachers, rooms, courses, and student sections.

1.4 Main Features

- Manage teachers, courses, rooms, sections, and time slots.
- Add holidays and unavailable days.
- Add teacher availability and preferences.
- Define hard constraints and soft preferences.
- Automatically generate a clash-free timetable.
- Check room capacity and room availability.
- Reduce teacher, room, and student-section conflicts.
- Prefer nearby rooms or classes when selected.
- Support energy-saving timetable options.
- Support early-ending or morning-class preferences.
- Allow manual timetable edits.
- Lock fixed classes so they are not changed.

- Re-optimize timetable after teacher or room changes.
- Show timetable quality, conflicts, and warnings.
- Publish timetable for teachers and students.
- Export or print the final timetable.

1.5 System Dependencies

The system depends on an internet connection, a web browser, a backend server, and a database to store teachers, courses, rooms, sections, timeslots, holidays, constraints, and generated timetables. It may also depend on university-provided data such as teacher availability, room details, academic calendar, and student section information, but it does not require integration with a full university ERP in the first version.

2. Process Model

2.1 Selected Model

The Incremental Process Model has been selected for the development of the University Timetable Optimization System (UTOS), with evolutionary prototyping used as a supporting technique within early increments to validate the data model, the constraint configuration interface, and the timetable rendering before full generation logic is built.

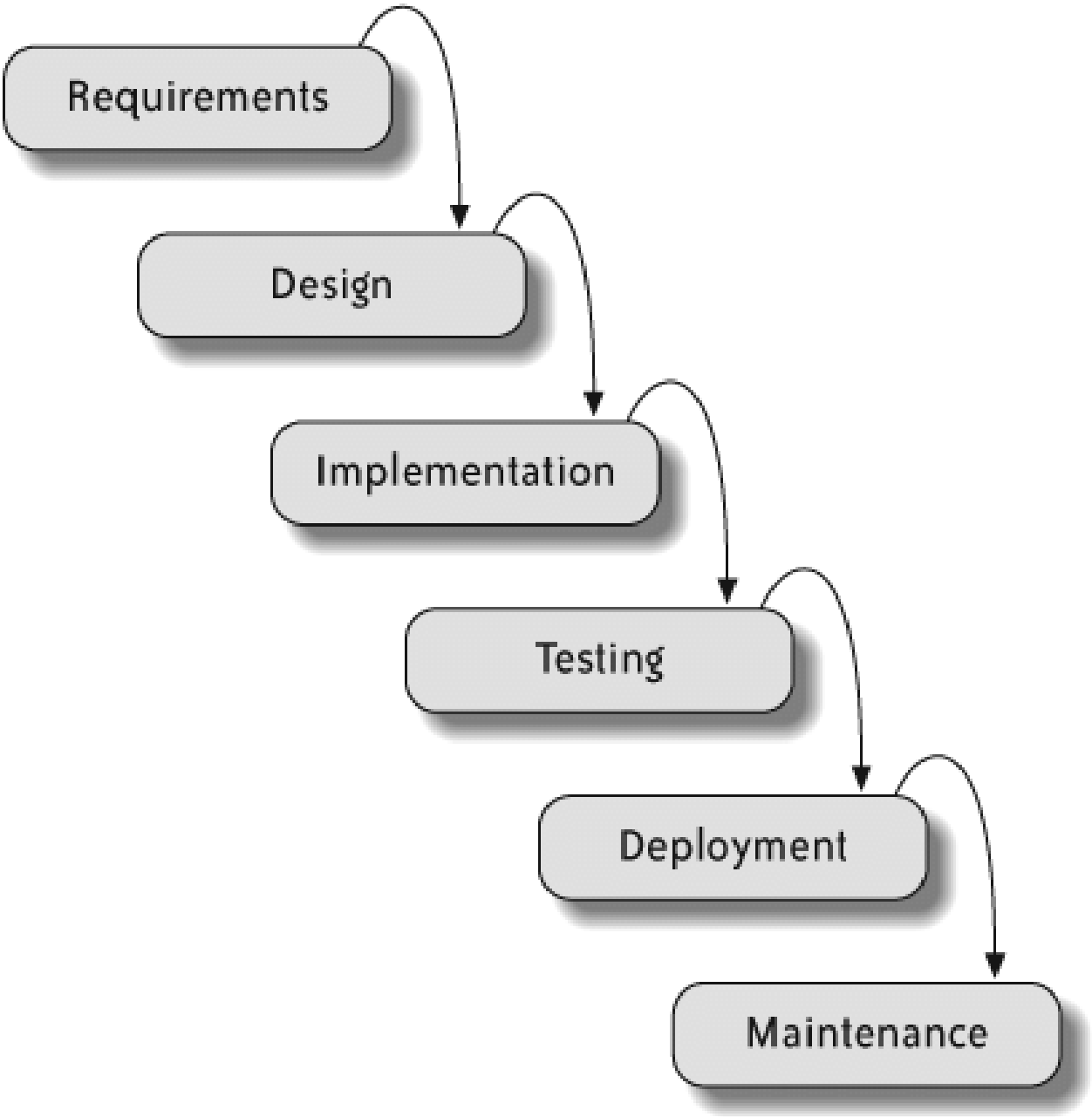
2.2 Why Incremental was Chosen

UTOS is not a simple CRUD project. The correct soft-preference weights for this university cannot be elicited through interviews; they emerge by running generation against real data and reviewing outputs. Incremental delivery places a working generator in front of stakeholders by Increment 3, which is precisely when this discovery becomes possible. The solver is also the highest-risk component of the system, and a linear lifecycle would defer its implementation until the end of the project, exposing algorithmic risk too late. Incremental development surfaces this risk early, when there is still room to respond.

Each increment ends with software that delivers value to a real user, not just internal scaffolding. Increment 1 gives the Admin a digital course catalog. Increment 3 gives a working timetable generator. Increment 5 gives change-repair capability. By Increment 6, the full system is in production use. This phased delivery also avoids the integration-end-of-project death spiral typical of waterfall projects of this complexity.

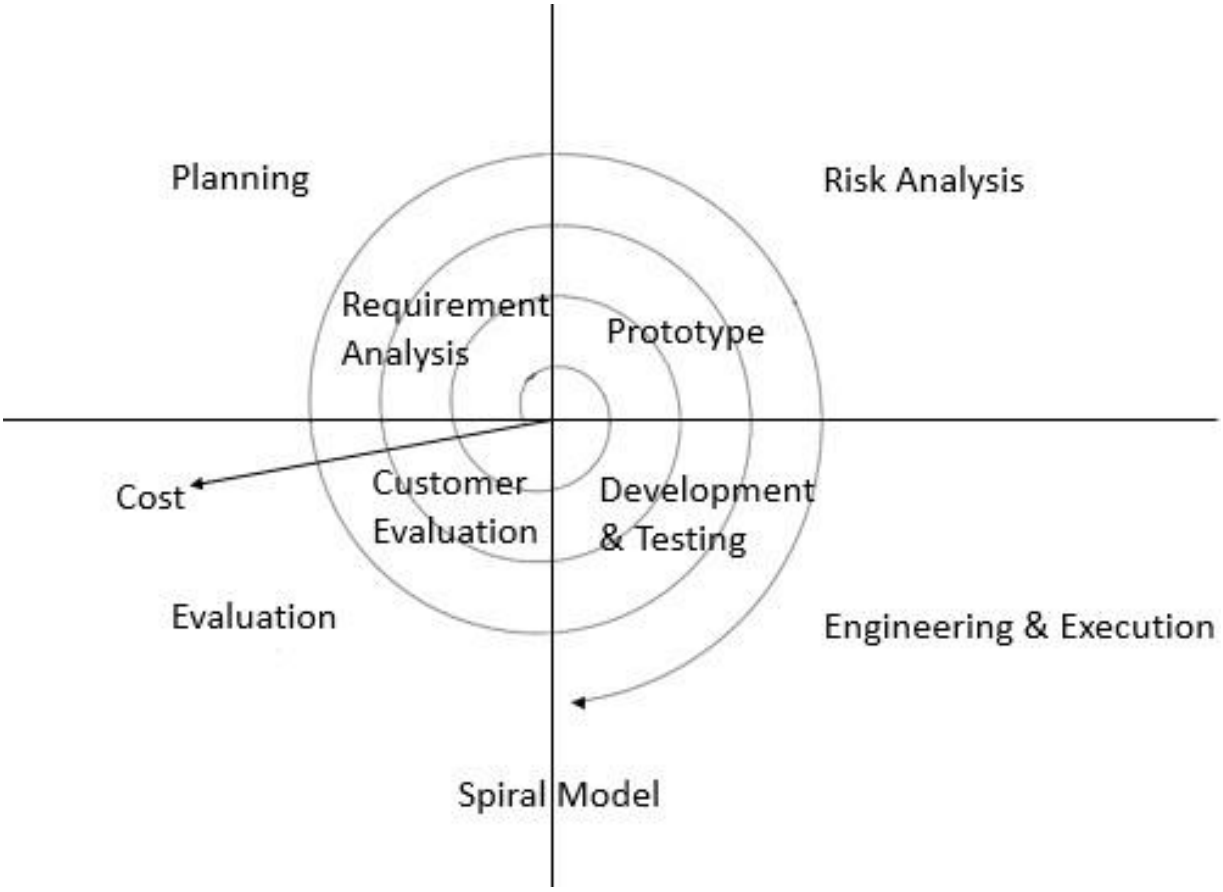
2.3 Why Other Models Were Rejected

Waterfall



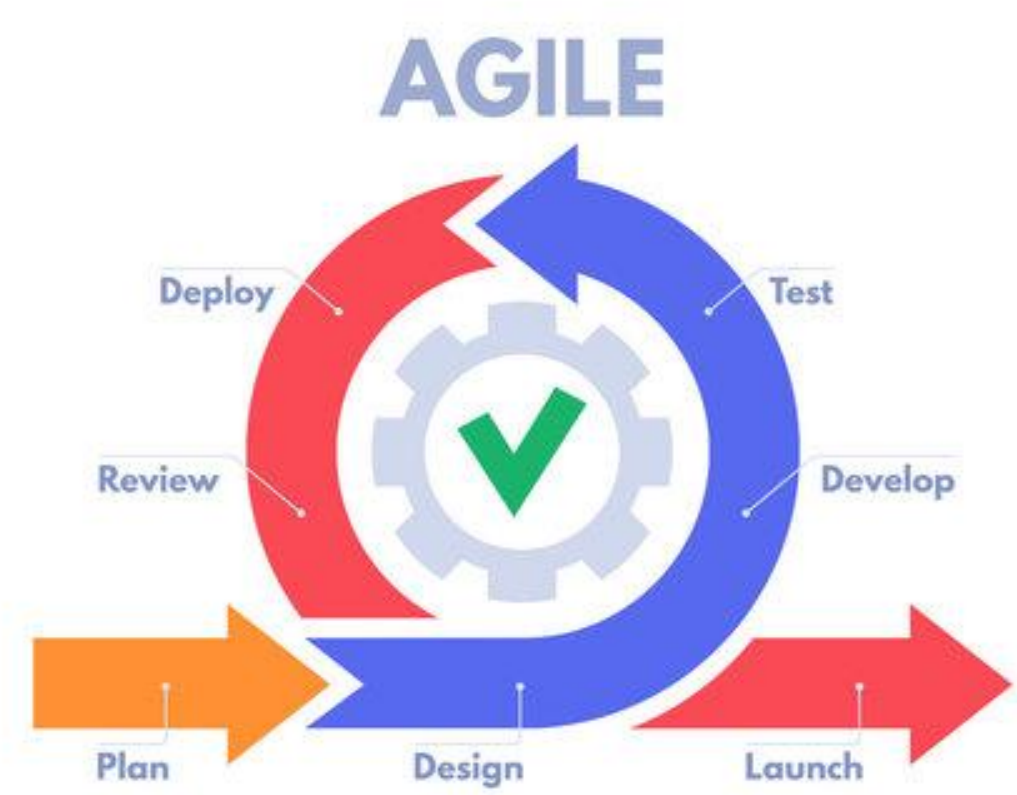
Waterfall demands a complete requirements specification before implementation begins. For a project where constraint weights are only correctable after viewing draft outputs, this guarantees a final system that misses real institutional needs. Late-stage changes such as a teacher's revised availability or a holiday added mid-semester become disproportionately expensive.

Spiral



The risk-driven structure of Spiral is correct in spirit, but the formal risk-analysis overhead is built for large systems. A small student team cannot sustain Spiral's ceremony, and the project's failure mode is a poor schedule, not loss of life or money. The project is too small for Spiral and too uncertain for Waterfall.

Pure Agile



Full Agile/Scrum adds organizational overhead (sprint planning, daily standups, retrospectives, product backlog grooming) that is not necessary for a project with a relatively stable core domain. Incremental delivery provides the adaptability of Agile within a cleaner, phased structure.

2.4 Increment Plan

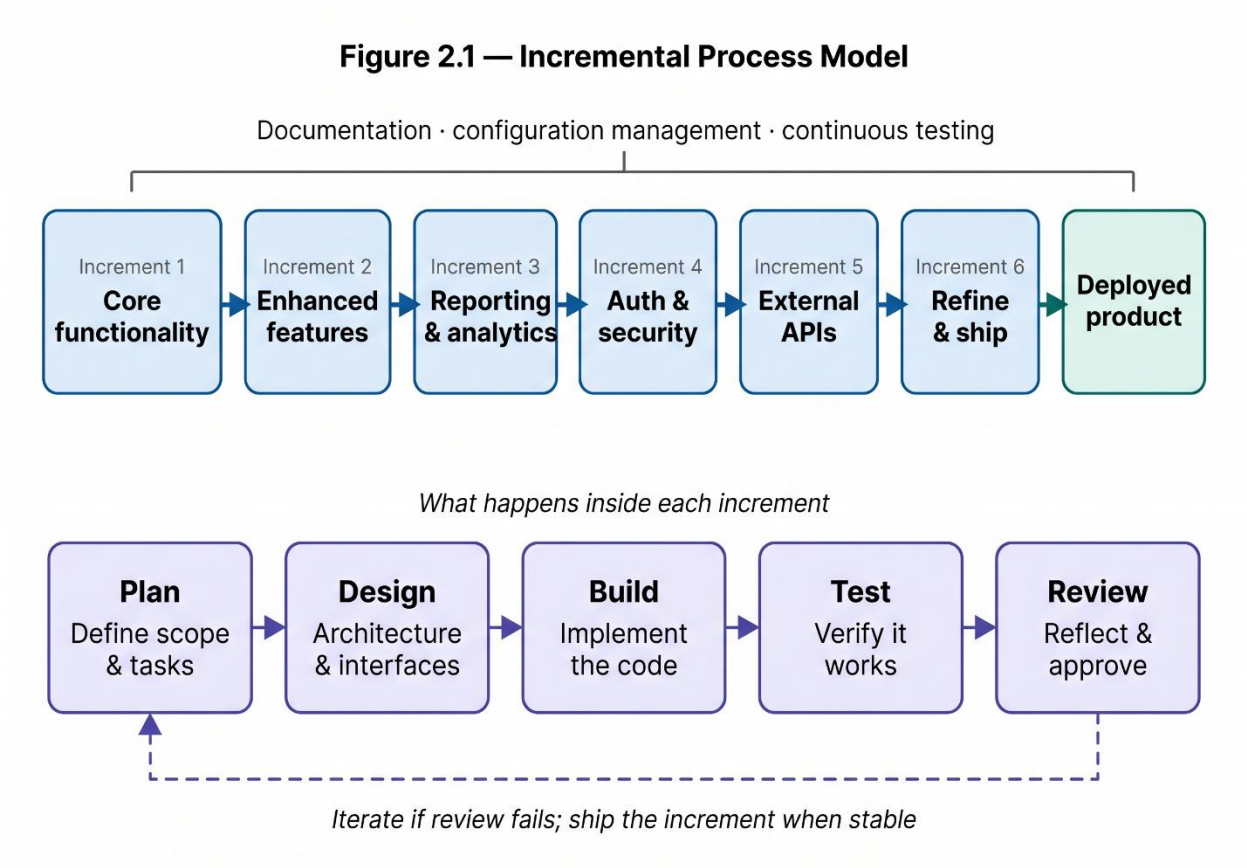
Increment	Focus	User-Visible Value
1	Master Data + Calendar	Admin can digitally maintain courses, teachers, rooms, and the academic calendar.
2	Constraints & Preferences	Admin can configure hard constraints and weighted soft preferences through a UI.
3	Generation + Conflicts	Admin can produce a draft timetable and inspect violations.
4	Manual Editing + Locking	Admin can override solver decisions and protect them from re-runs.

5	Repair / Re-optimization	Admin can apply teacher and room changes with minimum disruption.
6	Publishing + Role Views	Teachers and students see personal timetables; export to PDF/CSV.

2.5 Process Model Diagram

The Incremental Process Model diagram below shows six increments arranged left-to-right, each consisting of an inner mini-cycle of plan, design, build, test, and review. A horizontal arrow connects each increment to the next, indicating that the next increment begins with the working output of the previous one. A bracket spanning all six increments at the top represents documentation, configuration management, and continuous testing running throughout the project. Prototyping is applied as a supporting technique on Increments 1 and 2.

Figure 2.1 — Incremental Process Model



3. Requirements

3.1 Functional Requirements

UC01 — Maintain Academic Calendar

- The system shall allow the Timetable Administrator to add semester start and end dates.
- The system shall allow the Timetable Administrator to add holidays, blackout dates, and special closure days.
- The system shall prevent classes from being scheduled on holidays or unavailable dates.

UC02 — Manage Master Data

- The system shall allow the Timetable Administrator to add, update, and delete teachers.
- The system shall allow the Timetable Administrator to add, update, and delete courses, sections, rooms, and timeslots.
- The system shall store room details such as room number, capacity, building, floor, and available facilities.

UC03 — Define Constraints and Preferences

- The system shall allow the Timetable Administrator to define hard constraints such as teacher clashes, room clashes, and section clashes.
- The system shall allow the Timetable Administrator to define soft preferences such as early ending, nearby rooms, morning classes, and energy-saving options.
- The system shall allow preferences to be enabled, disabled, or given priority according to university needs.

UC04 — Generate Timetable

- The system shall generate a timetable based on teachers, courses, rooms, sections, timeslots, holidays, and constraints.
- The system shall prevent two teachers, rooms, or student sections from being assigned to conflicting classes at the same time.
- The system shall assign rooms according to class size, room capacity, and room availability.

UC05 — Review Timetable Quality

- The system shall display the generated timetable in a clear weekly view.
- The system shall show conflicts, warnings, and violated preferences in the generated timetable.

- The system shall show timetable quality details such as room usage, teacher load, and number of clashes.

UC06 — Request Teacher or Room Change

- The system shall allow a teacher to submit a timetable change request.
- The system shall allow the Department Coordinator to request changes for teachers, rooms, or class timings.
- The system shall store the reason and status of each change request.

UC07 — Re-optimize Timetable After Change

- The system shall allow the Timetable Administrator to re-optimize the timetable after a teacher, room, or timing change.
- The system shall try to minimize disturbance to already scheduled classes during re-optimization.
- The system shall keep locked classes unchanged during re-optimization.

UC08 — Lock Manual Decisions

- The system shall allow the Timetable Administrator to lock specific classes, rooms, or timeslots.
- The system shall prevent locked timetable entries from being changed automatically.
- The system shall allow the Timetable Administrator to unlock a timetable entry when needed.

UC09 — Publish Timetable

- The system shall allow the Timetable Administrator to publish the final approved timetable.
- The system shall make the published timetable visible to teachers and students.
- The system shall allow the final timetable to be exported or printed.

UC10 — View Personal Timetable

- The system shall allow teachers to view their own weekly timetable.
- The system shall allow students to view their section timetable.
- The system shall display class day, time, room, course, and teacher details in the timetable.

UC11 — View Resource Reports

- The system shall allow the Timetable Administrator to view room utilization reports.
- The system shall allow the Facility Manager to check which rooms are used, free, or overused.

- The system shall show reports related to room capacity, building usage, and peak timetable load.

UC12 — Manage Users and Roles

- The system shall allow the System Administrator to create user accounts for administrators, teachers, coordinators, students, and facility managers.
- The system shall assign roles and permissions to each user type.
- The system shall prevent unauthorized users from accessing restricted timetable management features.

3.2 Non-Functional Requirements

Performance

- The system shall load normal pages such as dashboard, timetable view, and forms within 3 seconds under normal usage.
- The system shall save added or updated data such as teachers, courses, rooms, and timeslots within 2 seconds.
- The system shall generate a basic timetable within 5–10 minutes, depending on the number of teachers, rooms, courses, sections, and constraints.
- The system shall display conflicts and timetable warnings within 5 seconds after generation.

Security

- The system shall allow only authorized users to log in using valid credentials.
- The system shall give different access rights to different users, such as Admin, Department Coordinator, Teacher, Student, Facility Manager, and System Administrator.
- The system shall allow only the Timetable Administrator to generate, edit, lock, re-optimize, and publish timetables.
- The system shall allow teachers and students to view only their relevant timetable information.
- The system shall prevent unauthorized users from changing timetable data, constraints, rooms, courses, or teacher information.

Usability

- The system shall provide a simple and clear interface for adding teachers, courses, rooms, sections, timeslots, holidays, and constraints.
- The system shall display the timetable in a weekly view that is easy for teachers, students, and administrators to understand.

- The system shall show clear error messages when clashes, missing data, or invalid entries are found.
- The system shall allow users to filter timetables by teacher, section, room, course, or day.
- The system shall use understandable labels and buttons so users can operate the system without advanced technical knowledge.

Availability

- The system shall be available during normal university working hours for timetable creation, editing, and viewing.
- The system shall allow teachers and students to view published timetables outside working hours when the server is online.
- The system shall store timetable data in a database so that information is not lost after logout or page refresh.
- The system shall provide basic backup support to recover timetable data in case of system failure.
- The system shall minimize downtime during timetable preparation and publishing periods.

3.3 Constraints

- The system shall be developed as a web-based application accessible through a standard web browser.
- The system shall use a database to store teachers, courses, rooms, sections, timeslots, holidays, constraints, and timetables.
- The system shall work on normal university computers without requiring high-end hardware.
- The system shall require an internet connection or local network connection to access the website.
- The system shall be completed within the given semester or project deadline.
- The system shall be developed within a student project budget, using free or low-cost tools where possible.
- The system shall support the timetable requirements of one department or limited university scope in the first version.
- The system shall not require integration with a full university ERP system in the initial version.
- The system shall depend on accurate input data from the university, such as teacher availability, room capacity, courses, sections, and academic calendar.

- The system shall be designed so that future features can be added later without rebuilding the whole system.

4. Use Cases

4.1 Actors

Actor	Role and Responsibilities
Timetable Administrator	Primary actor. Enters master data, defines constraints, generates timetables, fixes clashes, locks important classes, approves changes, and publishes the final timetable.
Department Coordinator	Manages department-level courses, sections, teachers, and preferences, and reviews whether the generated timetable is suitable.
Teacher	Views their personal timetable, submits availability or preferences, and requests changes if they are unavailable.
Student	Views their section timetable to know class time, room, teacher, and day.
Facility Manager	Checks room usage, room availability, capacity, and whether rooms are being used efficiently.
System Administrator	Manages user accounts, roles, permissions, system settings, backups, and basic security.

4.2 High-Level Use Cases

UC01 — Maintain Academic Calendar

Use Case ID	UC01
Name	Maintain Academic Calendar
Actors	Timetable Administrator
Type	Primary
Description	The administrator enters the working calendar for timetable generation, including semester dates, working days, holidays, blackout dates, exam weeks, and special closure days.
Essential	Define the working calendar so the timetable engine knows which dates are valid and which must be excluded.

Real	The administrator opens the calendar module, enters semester start and end dates, marks national holidays and university closures, and saves the calendar. The system then prevents any class from being scheduled on those dates.

UC02 — Manage Master Data

Use Case ID	UC02
Name	Manage Master Data
Actors	Timetable Administrator, Department Coordinator
Type	Primary
Description	The administrator records the fixed information required for scheduling, including departments, programs, courses, sections, teachers, rooms, buildings, capacities, room features, and timeslots.
Essential	Maintain the entities that the timetable depends on so the solver has clean, validated input data.
Real	The administrator opens each master-data form (e.g. Courses), adds new records or edits existing ones, and the system validates that no course exists without a teacher and that room capacities are positive.

UC03 — Define Constraints and Preferences

Use Case ID	UC03
Name	Define Constraints and Preferences
Actors	Timetable Administrator, Department Coordinator
Type	Primary
Description	The administrator specifies scheduling rules and policy priorities, defining hard constraints (no teacher conflicts, no room conflicts, respect unavailable periods) and soft preferences (compact schedules, room proximity, early finishing, morning preference, room stability, traffic reduction, energy-aware grouping).
Essential	Configure the rulebook the solver follows when assigning classes to rooms and timeslots.
Real	The administrator opens the constraints module, toggles which preferences are active, sets priority weights, and the system saves the configuration for use during the next generation run.

UC04 — Generate Timetable

Use Case ID	UC04
Name	Generate Timetable
Actors	Timetable Administrator
Type	Primary
Description	The administrator triggers timetable generation. The system assigns every class session to a room and timeslot while satisfying all hard constraints and minimizing soft constraint penalties.
Essential	Produce a feasible timetable that satisfies hard rules and optimizes soft goals.
Real	The administrator selects the academic term, clicks Generate, and the system queues the solver. The frontend shows a progress indicator while polling status. When complete, the administrator is taken to the conflict report.

UC05 — Review Timetable Quality

Use Case ID	UC05
Name	Review Timetable Quality
Actors	Timetable Administrator, Department Coordinator
Type	Primary
Description	The system displays the generated timetable, conflict summary, score breakdown, room usage, teacher load, and violated preferences. The administrator reviews these and decides whether the result is acceptable or whether the timetable needs adjustment.
Essential	Inspect the quality of the generated schedule before publishing.
Real	The administrator opens the review screen, sees a list of hard violations (e.g. "Teacher Dr. Ahmad double-booked") and the soft penalty score. They click each violation to see affected classes and decide on next steps.

UC06 — Request Teacher or Room Change

Use Case ID	UC06
Name	Request Teacher or Room Change
Actors	Teacher, Department Coordinator
Type	Primary

Description	A teacher or coordinator submits a request to change a timetable assignment due to leave, room issues, new preference, or unexpected events. The system stores the request with its reason and status for review.
Essential	Submit a timetable modification request that the administrator can act on.
Real	The teacher logs in, opens their personal timetable, selects an affected class, fills in the reason, and submits. The system stores the request as pending and notifies the administrator.

UC07 — Re-optimize Timetable After Change

Use Case ID	UC07
Name	Re-optimize Timetable After Change
Actors	Timetable Administrator
Type	Primary
Description	After accepting a change request, the administrator asks the system to update the timetable while minimizing disruption to already scheduled classes and preserving as much of the accepted timetable as possible.
Essential	Repair the timetable with minimum disruption to other classes.
Real	The administrator selects an approved change request, clicks Re-optimize, and the system runs the solver in repair mode. The result preserves locked classes and minimizes the number of classes moved.

UC08 — Lock Manual Decisions

Use Case ID	UC08
Name	Lock Manual Decisions
Actors	Timetable Administrator
Type	Primary
Description	The administrator marks specific classes, rooms, or time assignments as locked. Locked entries are protected from being changed automatically by future re-optimization runs.
Essential	Protect selected timetable assignments so they survive re-optimization.
Real	The administrator selects a class on the timetable grid, clicks Lock, and the system marks it as immutable. Future solver runs treat it as a fixed constraint.

UC09 — Publish Timetable

Use Case ID	UC09
Name	Publish Timetable
Actors	Timetable Administrator
Type	Primary
Description	Once the timetable is finalized and approved, the administrator publishes it. The published timetable becomes visible to teachers and students and can be exported in printable or downloadable form.
Essential	Release the approved timetable to all users.
Real	The administrator reviews the final draft, clicks Publish, and the system marks the timetable as live and stores a versioned snapshot. Teachers and students immediately see the published version on their dashboards.

UC10 — View Personal Timetable

Use Case ID	UC10
Name	View Personal Timetable
Actors	Teacher, Student
Type	Primary
Description	A teacher or student opens the system and sees the published timetable filtered to their own classes. The view shows day, time, room, course, and teacher details.
Essential	Access the final schedule filtered to the user's relevant classes.
Real	The user logs in, the system identifies their role and identity, and the dashboard shows only the classes they are assigned to or enrolled in. They can filter by day or week.

UC11 — View Resource Reports

Use Case ID	UC11
Name	View Resource Reports
Actors	Facility Manager, Timetable Administrator
Type	Primary

Description	The system provides reports on room utilization, peak traffic periods, late-day load, compactness, and energy-related scheduling patterns for administrative review.
Essential	Evaluate the operational impact of the timetable.
Real	The facility manager opens the reports section and selects "Room Utilization". The system generates a per-room usage table showing which rooms are over- or underused across the week.

UC12 — Manage Users and Roles

Use Case ID	UC12
Name	Manage Users and Roles
Actors	System Administrator
Type	Primary
Description	The system administrator creates and manages user accounts, assigns roles and permissions, and ensures that unauthorized users cannot access restricted features.
Essential	Maintain user accounts and access control for the entire system.
Real	The system administrator creates a new account, assigns the role (e.g. Teacher or Coordinator), and the user receives login credentials. The system enforces role-based access on all subsequent requests.

4.3 Use Case Diagram

The use case diagram below shows the major actors and the high-level functions of the University Timetable Optimization System. The Admin (Timetable Administrator) is the primary actor and connects to most administrative use cases. The Department Head helps set constraints and reviews timetables. Teachers can provide preferences and view their personal timetable, while Students can view the published timetable for their section.

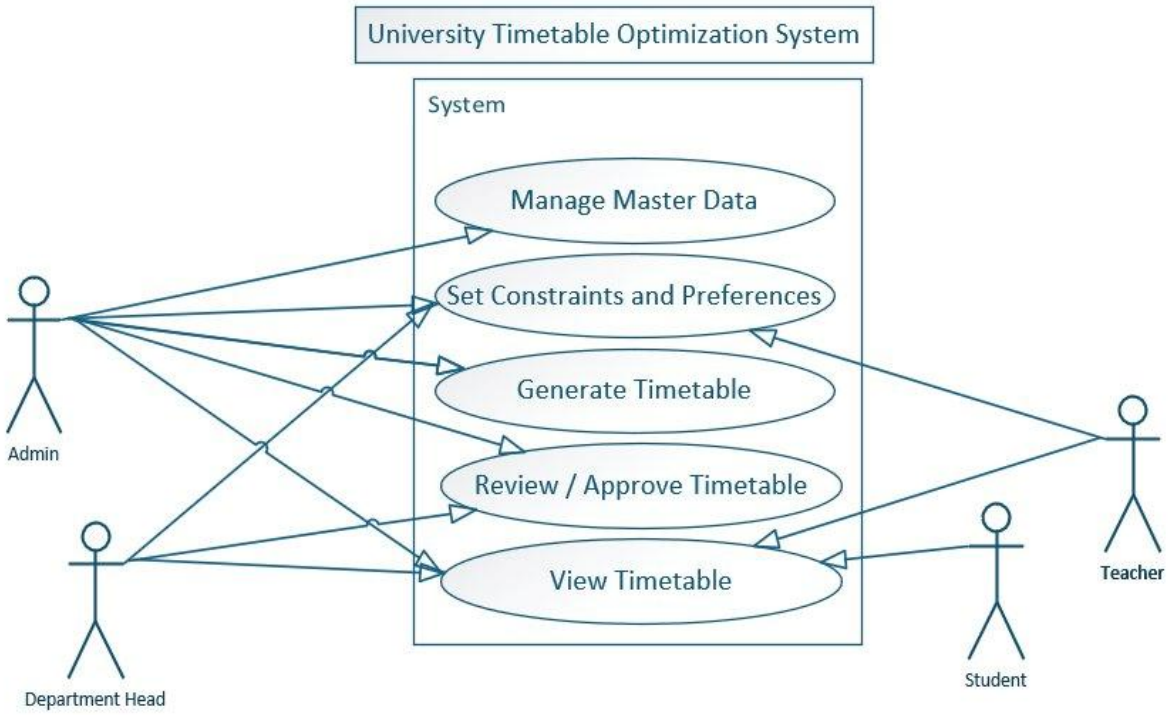


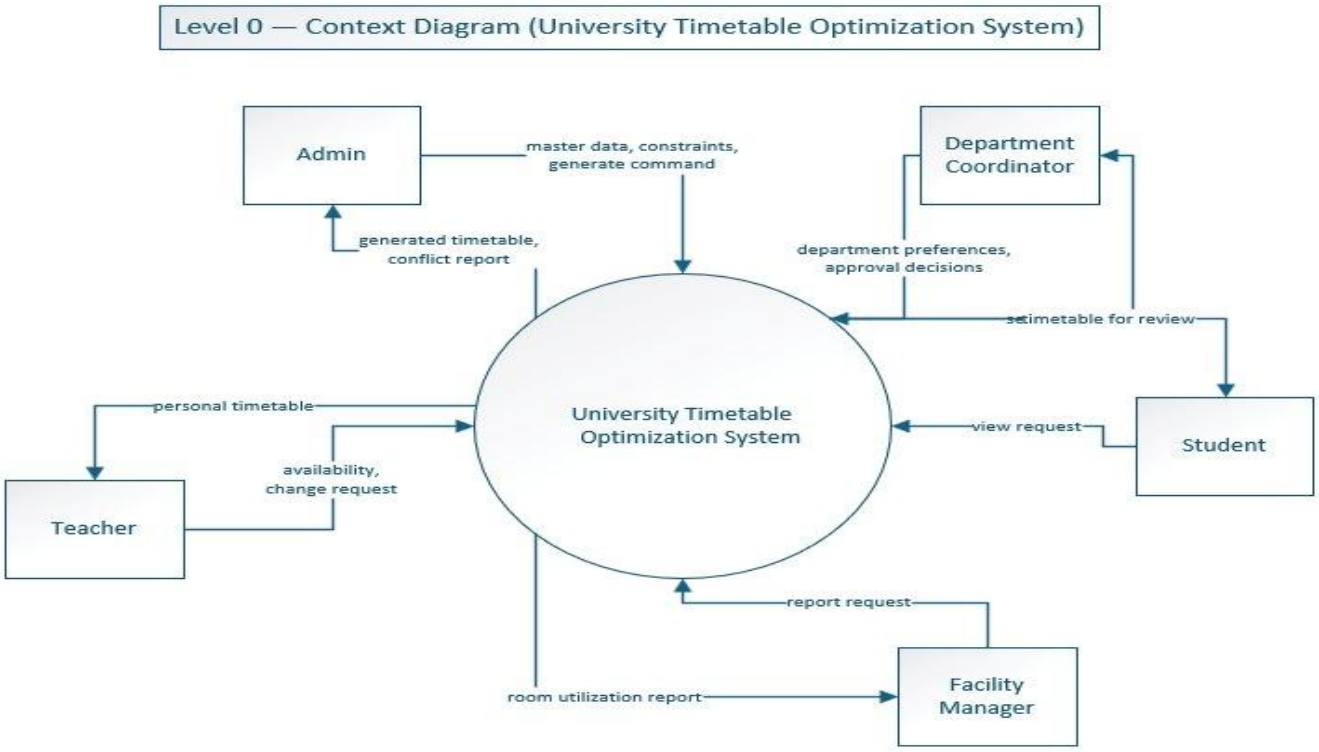
Figure 4.1 — Use Case Diagram of the University Timetable Optimization System

5. System Modeling

5.1 DFD Level 0 — Context Diagram

The Level 0 Context Diagram represents the entire University Timetable Optimization System as a single process and shows the data flowing in and out across the system boundary. The five external entities are the Admin, Department Coordinator, Teacher, Student, and Facility Manager. The Admin provides master data, constraints, and the generate command, and receives the generated timetable and conflict report. The Department Coordinator provides department preferences and approval decisions and receives the timetable for review. Teachers submit availability and change requests and receive their personal timetable. Students send view requests and receive their section timetable. The Facility Manager sends report requests and receives room utilization reports.

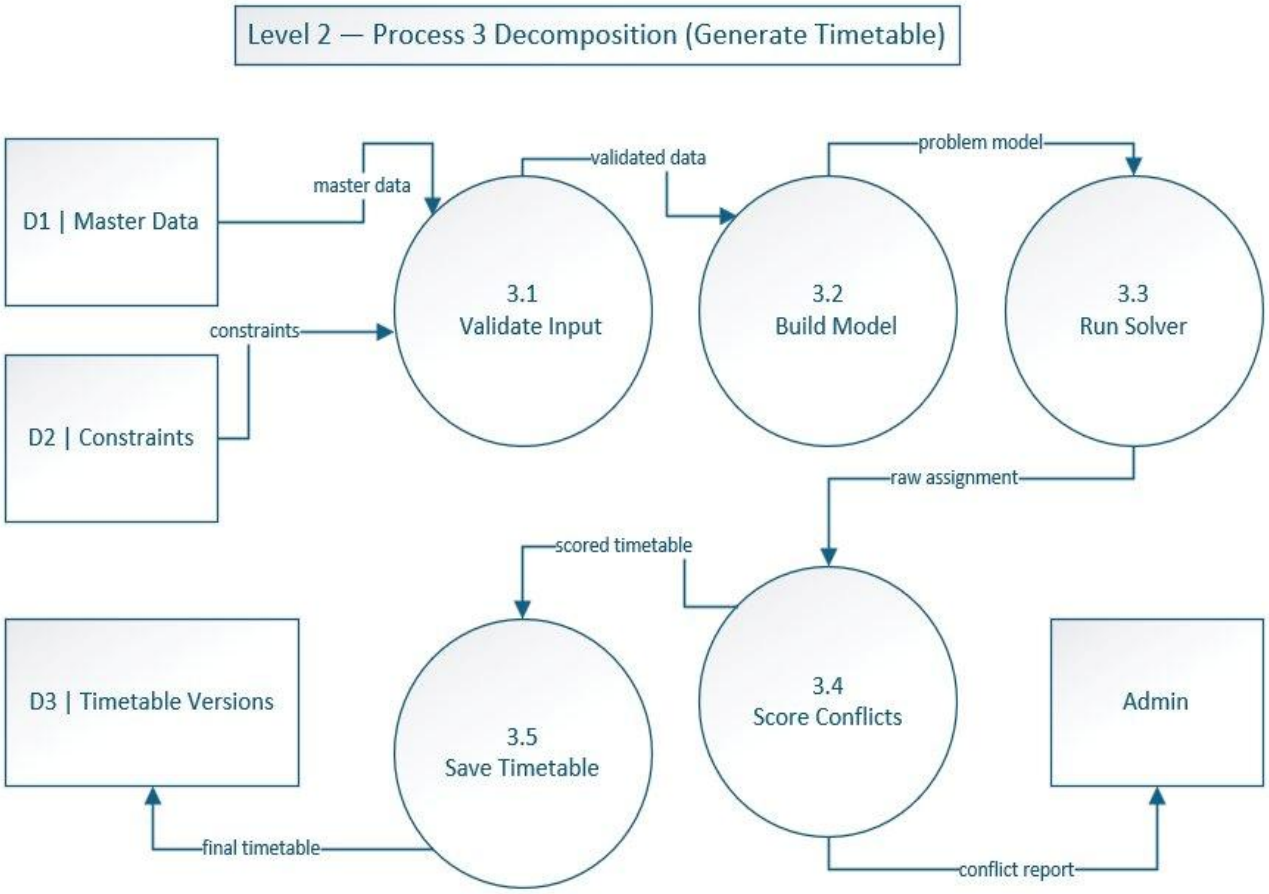
Figure 5.1 — Level 0 Context Diagram



5.2 DFD Level 1

The Level 1 DFD decomposes the system into six major processes and four data stores. Process 1 (Manage Master Data) receives course, teacher, and room data from the Admin and stores them in D1 (Master Data). Process 2 (Configure Constraints) receives institutional rules from the Department Coordinator and stores them in D2 (Constraints and Preferences). Process 3 (Generate Timetable) is the core of the system: it reads master data from D1 and active constraints from D2, generates a timetable, writes it to D3 (Timetable Versions), and returns a conflict report to the Admin. Process 4 (Handle Change Requests) receives change requests from Teachers, stores them in D4 (Change Requests), and updates the timetable in D3 after re-optimization. Process 5 (Publish & View Timetable) reads the published timetable from D3 and delivers personal timetables to Teachers and section timetables to Students. Process 6 (Generate Reports) combines master data from D1 and timetable data from D3 to produce utilization reports for the Facility Manager.

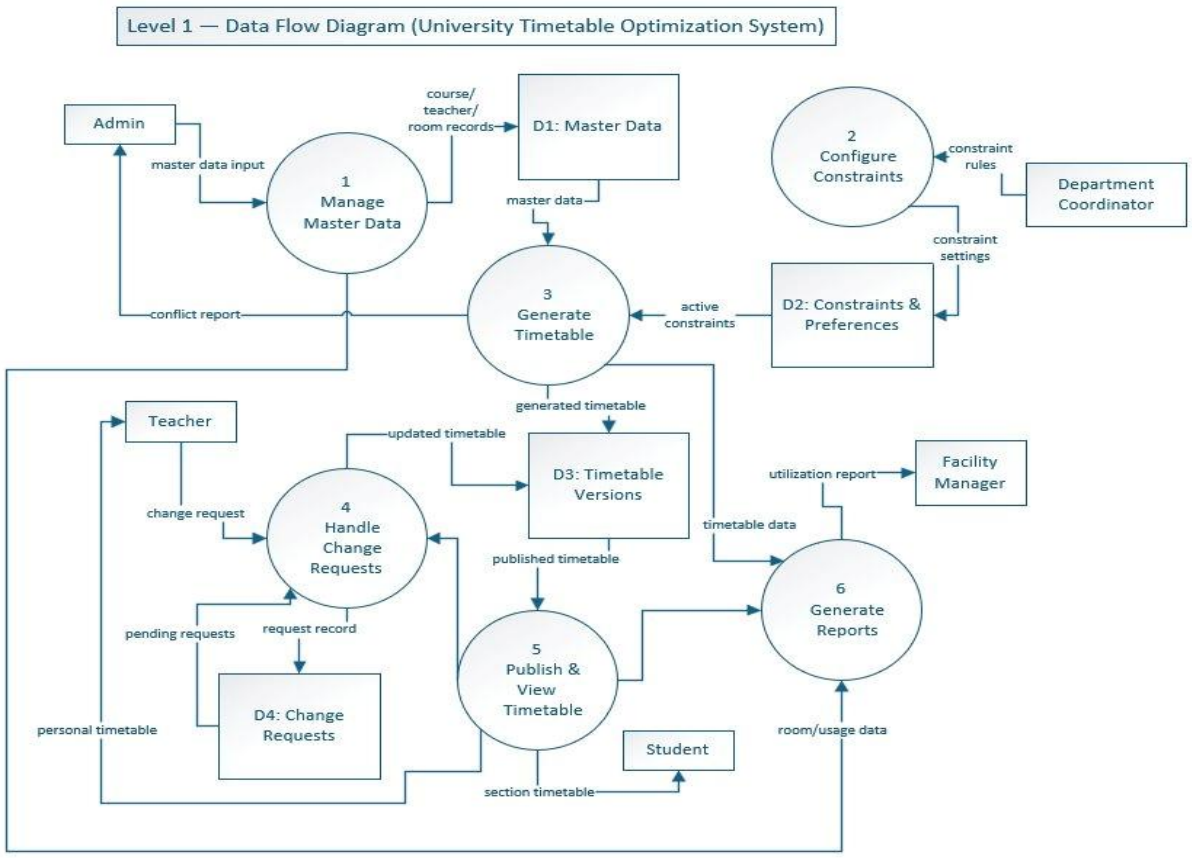
Figure 5.2 — Level 1 Data Flow Diagram



5.3 DFD Level 2 — Generate Timetable Decomposition

The Level 2 DFD decomposes Process 3 (Generate Timetable) into five sub-processes. Sub-process 3.1 (Validate Input) receives master data from D1 and constraints from D2 and verifies that all required information is present and consistent. Sub-process 3.2 (Build Model) converts the validated data into a solver-ready problem model containing lectures, rooms, timeslots, and constraint expressions. Sub-process 3.3 (Run Solver) executes the constraint satisfaction algorithm and produces a raw assignment. Sub-process 3.4 (Score Conflicts) evaluates the result against hard and soft rules, computes a penalty score, and returns a conflict report to the Admin. Sub-process 3.5 (Save Timetable) persists the final timetable into D3 (Timetable Versions). This decomposition makes the most uncertain part of the system explicit and shows that the solver is only one stage in a multi-step pipeline.

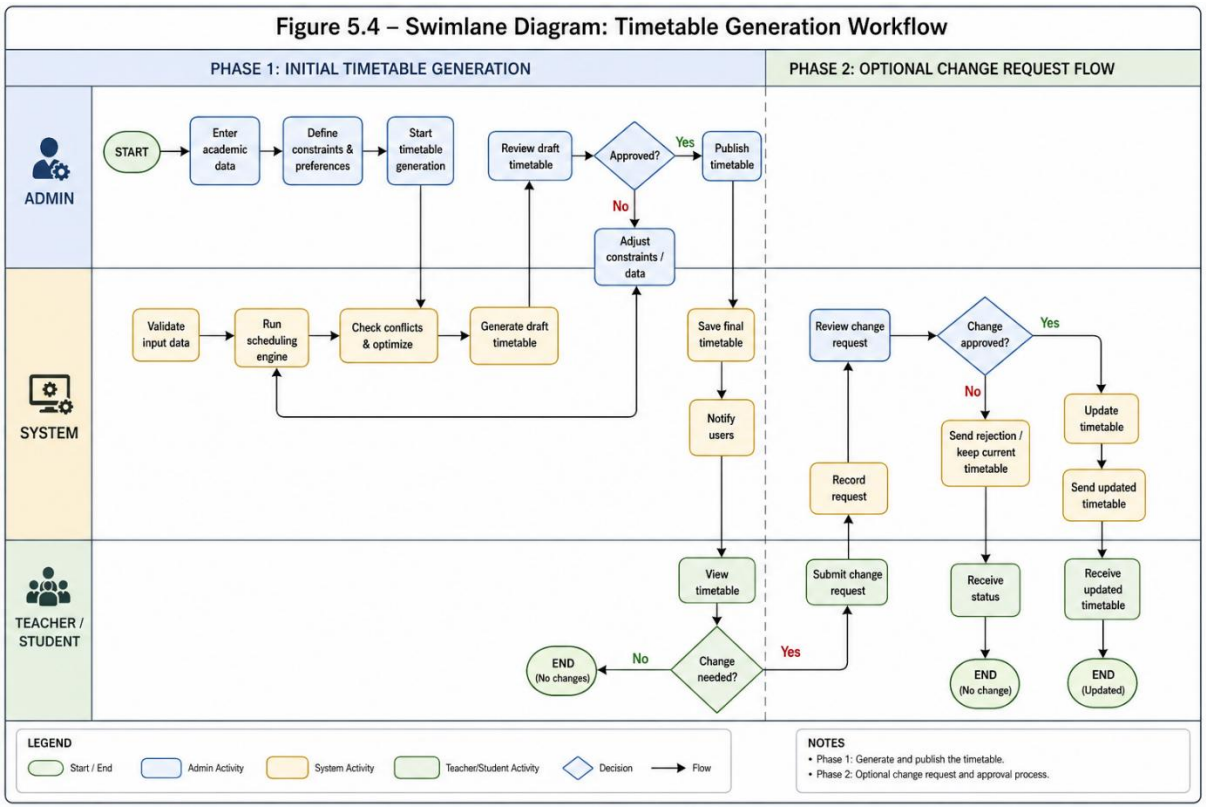
Figure 5.3 — Level 2 DFD: Process 3 Decomposition



5.4 Swimlane Diagram

The swimlane diagram below shows the timetable generation workflow distributed across the three primary actors: Admin, System, and Teacher/Student. The Admin lane includes the activities of entering master data, defining constraints, triggering generation, reviewing conflicts, and publishing. The System lane includes validating data, running the solver, detecting conflicts, and storing the timetable. The Teacher/Student lane includes viewing the published timetable and submitting change requests. Arrows between lanes show handoffs of responsibility, such as the Admin clicking Generate (handoff from Admin to System) and the System returning the conflict report (handoff from System to Admin).

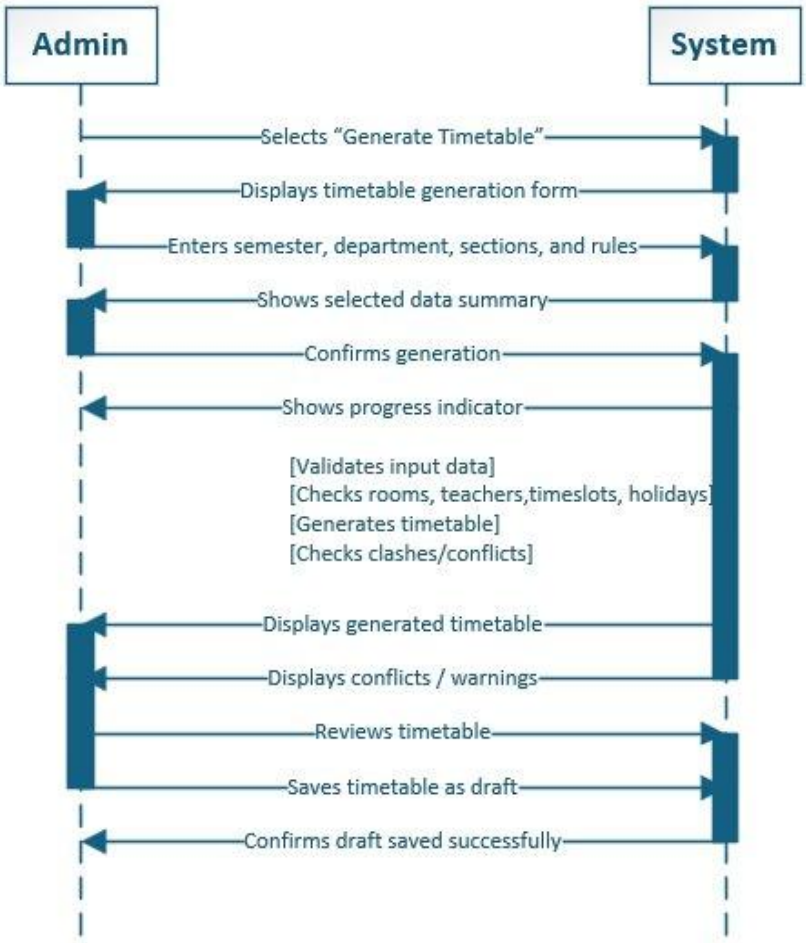
Figure 5.4 — Swimlane Diagram



5.5 System Sequence Diagram — Generate Timetable

The system sequence diagram below shows the message exchange between the Admin and the System for the Generate Timetable use case (UC04). The Admin selects "Generate Timetable", and the System displays the generation form. The Admin enters the semester, department, sections, and rules, and the System shows a data summary. After confirmation, the System shows a progress indicator and internally validates input data, checks rooms, teachers, timeslots, and holidays, generates the timetable, and checks for clashes and conflicts. The System then displays the generated timetable, the conflicts, and warnings. The Admin reviews the timetable and saves it as a draft, and the System confirms successful saving.

Figure 5.5 — System Sequence Diagram for UC04 Generate Timetable

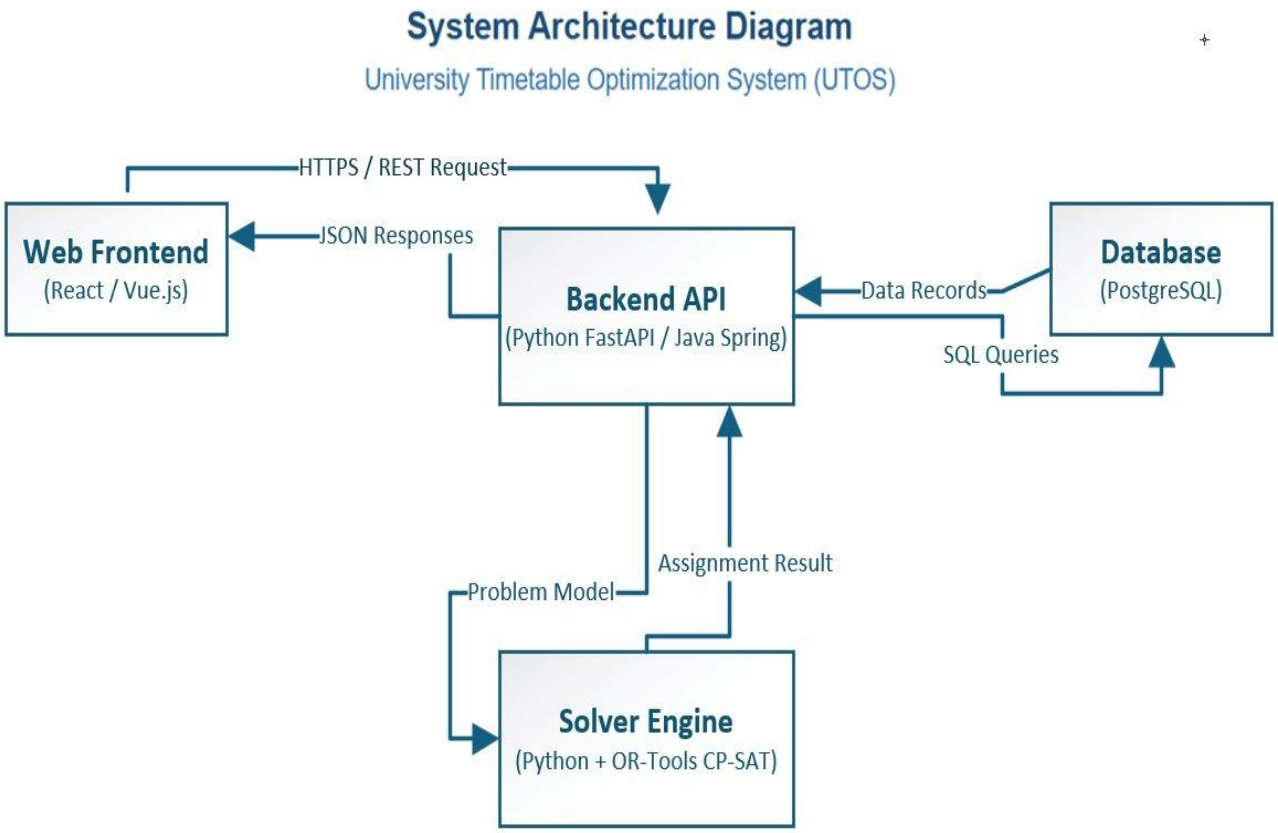


5.6 Architecture Diagram

The system follows a four-tier architecture with a clear separation of concerns. The Web Frontend (built with React or Vue.js) runs in the user's browser and communicates with the Backend API over HTTPS using REST endpoints. The Backend API (Python FastAPI or Java Spring Boot) handles authentication, business logic, request routing, and orchestration between the database and the solver. The Backend persists all master data, generated schedules, and conflict records in the PostgreSQL Database through standard SQL queries. When the Admin triggers timetable generation, the Backend constructs a structured problem model and sends it to the Solver Engine (Python with Google OR-Tools CP-SAT), which runs constraint satisfaction algorithms and returns an assignment result. The result is stored in the database and returned to the Frontend as JSON.

This separation allows each tier to be developed, tested, and upgraded independently. Most importantly, the Solver Engine sits behind a clean interface, so the underlying algorithm can be replaced (for example, from a greedy heuristic to a metaheuristic or a different solver library) without affecting the Frontend, the database schema, or the authentication layer.

Figure 5.6 — System Architecture Diagram



6. Conclusion

This document specifies the University Timetable Optimization System (UTOS), a web-based platform that generates, manages, and repairs class schedules under multiple competing objectives. The system models the university timetabling problem as a constraint satisfaction task with hard rules (no clashes, room capacity respected) and configurable soft preferences (room proximity, energy-aware compaction, morning preference, early finish, minimal disruption after changes).

The Incremental Process Model was selected because the project's defining characteristic, that correct constraint weights are discovered by running the system rather than by specifying them upfront, directly matches what incremental delivery is built to support. The system was decomposed into six increments, each delivering user-visible value rather than internal scaffolding. Twelve high-level use cases capture the major user goals across six actor roles, and the system modeling section presents the full picture: a Level 0 context diagram, a Level 1 data flow diagram with six processes and four data stores, a Level 2 decomposition of the most complex process, a swimlane diagram showing cross-actor workflow, a system sequence diagram for the core generation use case, and a four-tier architecture diagram with a decoupled solver.

The next phase of the project will focus on implementing Increment 1 (master data and academic calendar), defining the database schema, and prototyping the constraint authoring interface. Increment 3 (the first working timetable generator) is the most important milestone, as it will surface algorithmic and modeling risks while there is still room in the project to respond to them.

End of Document · University Timetable Optimization System · v1.0